

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import sqlite3
```

```
In [2]: !pip install seaborn
```

```
Requirement already satisfied: seaborn in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (2.5.3)
Requirement already satisfied: pandas>=1.2 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (3.0.5)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from seaborn) (3.11.2)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.0)
Requirement already satisfied: cycler>=0.10 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.65.0)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.1)
Requirement already satisfied: packaging>=20.0 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.3)
Requirement already satisfied: pillow>=9 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.3.0)
Requirement already satisfied: pyparsing>=3 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas>=1.2->seaborn) (2026.3)
Requirement already satisfied: six>=1.5 in C:\Users\admin\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
```

```
In [3]: #connection with database
conn = sqlite3.connect('customer_churn.db')

sql_query = """
    SELECT name
    FROM sqlite_master
    WHERE type= 'table'
    """

tables= pd.read_sql(sql_query, conn)
```

```
#create dataframe for each table
for table_name in tables['name']:
    df= pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"crreated dataframe: df_{table_name}")

conn.close
```

crreated dataframe: df_db_customer
 crreated dataframe: df_db_subscription
 crreated dataframe: df_db_support

Out[3]: <function Connection.close()>

```
In [4]: #print table name and column names
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name : {table_name}")
    #get column information
    columns_query = f"PRAGMA table_info({table_name});"
    columns = pd.read_sql(columns_query, conn)
    print("columns:")
    print(columns['name'].tolist())

conn.close()
```

Table Name : db_customer
 columns:
 ['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name : db_subscription
 columns:
 ['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']

Table Name : db_support
 columns:
 ['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']

```
In [5]: #data cleaning
```

```
In [6]: df_db_customer.tail()
```

```
Out[6]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead>
<tbody> <tr> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody>
<tbody></tbody>
```

0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

```
In [7]: df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   customerid  21 non-null    str
1   name        21 non-null    str
2   country     18 non-null    str
3   state       21 non-null    str
4   gender      21 non-null    str
5   dob         21 non-null    str
6   interests   4 non-null     str
7   pincode     0 non-null     object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

```
In [8]: #a.rename (name) column
#b. drop col interest and pincode
#c.change DOB datatype
#d. fix gender
#e. fix misising values..country
```

```
In [9]: #a.rename (name) column
df_db_customer.rename(columns = {'name' : 'customer_name'},inplace=True )
```

```
In [10]: #b. drop col interest and pincode
#df_db_customer.drop(df_db_customer.columns[-2:],axis=1)
#df_db_customer
df_db_customer.drop(columns=['interests', 'pincode'], inplace= True)
```

```
In [11]: df_db_customer
```

```
Out[11]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th>
<th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th>
<th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody>
</tbody>
```

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

```
In [12]: #c.change DOB datatype
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

```
In [13]: df_db_customer.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   customerid      21 non-null    str
 1   customer_name   21 non-null    str
 2   country         18 non-null    str
 3   state           21 non-null    str
 4   gender          21 non-null    str
 5   dob             21 non-null    datetime64[us]
dtypes: datetime64[us](1), str(5)
memory usage: 1.1 KB

```

```

In [14]: #d. fix gender
df_db_customer['gender'].unique()
df_db_customer['gender']=df_db_customer['gender'].replace({'Men': 'Male', 'Women' :

```

```

In [15]: #e. fix misising values..country
df_db_customer[df_db_customer['country'].isna()]

```

```

Out[15]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> </thead> <tbody> <th>5</th> <th>8</th>
<th>12</th> </tbody> <tbody></tbody>

```

0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10
0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01

```

In [16]: state_country_mapping=df_db_customer.dropna(subset=['country']).set_index('state')[
df_db_customer['country']= df_db_customer['country'].fillna(df_db_customer['state'])

```

```

In [17]: df_db_subscription.head()

```

```
Out[17]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th>
<th>renewal_date</th> <th>plan_type</th> <th>contract_type</th>
<th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th>
<th>cltv</th> <th>churn_score</th> </thead> <tbody> <th>0</th> <th>1</th>
<th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaN	NaN	13.99	627	12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaN	NaN	6.99	210	34
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaN	NaN	22.99	1725	8
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88

```
In [18]: df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                            21 non-null     str
1   subscription_start_date                21 non-null     str
2   subscription_type                      21 non-null     str
3   renewal_date                          21 non-null     str
4   plan_type                             21 non-null     str
5   contract_type                         21 non-null     str
6   cancellation_date                     6 non-null      str
7   cancellation_reason                    6 non-null      str
8   monthly_charges                       21 non-null     float64
9   cltv                                  21 non-null     int64
10  churn_score                           21 non-null     int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB
```

```
In [19]: #a. change to datetime dataframe
date_col= ['subscription_start_date', 'renewal_date', 'cancellation_date']
df_db_subscription[date_col]=df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customerid                           21 non-null     str
1   subscription_start_date               21 non-null     datetime64[us]
2   subscription_type                     21 non-null     str
3   renewal_date                         21 non-null     datetime64[us]
4   plan_type                            21 non-null     str
5   contract_type                        21 non-null     str
6   cancellation_date                    6 non-null      datetime64[us]
7   cancellation_reason                   6 non-null      str
8   monthly_charges                      21 non-null     float64
9   cltv                                 21 non-null     int64
10  churn_score                           21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB
```

In []:

In [20]:

df_db_support.head()

Out[20]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th></th> <th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody> <th>0</th><th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

In [21]:

#remove last 2 column

df_db_support.drop(columns= ['col_1', 'comment'], inplace= True)

In [22]:

df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])

df_db_support.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customerid            9 non-null     str
1   complaint_date        9 non-null     datetime64[us]
2   escalations           9 non-null     str
3   csat_score            9 non-null     int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 420.0 bytes
```

In []:

In []:

In [23]:

```
#New Topic
# Feature engineering and data analysis

#create a new col using existing col

df_db_subscription['churn_flag']=np.where(df_db_subscription['cancellation_date'].n
```

In [24]:

```
df_db_subscription.head()
```

Out[24]:

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th>
<th>renewal_date</th> <th>plan_type</th> <th>contract_type</th>
<th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th>
<th>cltv</th> <th>churn_score</th> <th>churn_flag</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>
```

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12	0
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34	0
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8	0
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1

In [25]:

```
df_db_subscription.shape
```

Out[25]:

```
(21, 12)
```

In [26]: *#This is just trail first fix table and then merge (real merge is down)*

```
df=(df_db_subscription
      .merge(df_db_customer, on= 'customerid', how= 'left')
      .merge(df_db_support, on= 'customerid', how= 'left'))
```

In [27]: df.shape

Out[27]: (23, 20)

In [28]: df_db_subscription['customerid'].nunique()

Out[28]: 21

In [29]: df_db_customer['customerid'].nunique()

Out[29]: 21

In [30]: df_db_support['customerid'].nunique()

Out[30]: 7

In [31]: df_db_support['customerid'].size

Out[31]: 9

In [32]: df_db_support

Out[32]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th>
<th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> </tbody> <tbody>
</tbody>

0003-MKNFE	2024-08-28	N	60
------------	------------	---	----

0003-MKNFE	2024-08-28	Y	10
------------	------------	---	----

0013-EXCHZ	2024-01-20	Y	20
------------	------------	---	----

0013-MHZWF	2025-03-18	N	90
------------	------------	---	----

0013-SMEOE	2024-11-01	N	30
------------	------------	---	----

0017-IUDMW	2024-04-10	Y	25
------------	------------	---	----

0019-EFAEP	2024-09-27	Y	30
------------	------------	---	----

0022-TCJCI	2024-09-13	Y	10
------------	------------	---	----

0022-TCJCI	2024-09-14	N	90
------------	------------	---	----

In [33]: `df_db_support['complaint_count']=df_db_support.groupby('customerid')['customerid'].`

In [34]: `df_db_support`

Out[34]: `<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> <th>complaint_count</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th>
<th>8</th> </tbody> <tbody> </tbody>`

0003-MKNFE 2024-08-28 N 60 2

0003-MKNFE 2024-08-28 Y 10 2

0013-EXCHZ 2024-01-20 Y 20 1

0013-MHZWF 2025-03-18 N 90 1

0013-SMEOE 2024-11-01 N 30 1

0017-IUDMW 2024-04-10 Y 25 1

0019-EFAEP 2024-09-27 Y 30 1

0022-TCJCI 2024-09-13 Y 10 2

0022-TCJCI 2024-09-14 N 90 2

In [35]: `df_db_support=df_db_support.sort_values('complaint_date').drop_duplicates('customer`

In [36]: `df=(df_db_subscription
 .merge(df_db_customer, on= 'customerid', how= 'left')
 .merge(df_db_support, on= 'customerid', how= 'left'))`

In [37]: `df.shape`

Out[37]: (21, 21)

In []:

In [38]: `df.to_csv('exported_churn_data.csv', index= False)`

In [39]: `df.columns`

Out[39]: Index(['customerid', 'subscription_start_date', 'subscription_type',
 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
 'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
 'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
 dtype='str')

In [40]: `#Data Analysis`

In [41]: *#1. churn Rate*

```
In [42]: churn_rate=df['churn_flag'].mean()*100
print("Churn rate = ", round(churn_rate,2), "%")
```

Churn rate = 28.57 %

In [43]: *#2.Retention Rate*

```
retention_rate= 100- churn_rate
print("Retention rate", round(retention_rate,2), "%")
```

Retention rate 71.43 %

In [44]: *#3.churn_rate_by_plan*

```
churn_by_flag=(df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).reset
print(churn_by_flag)
```

	plan_type	churn_rate_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

In [45]: *#4a. churn by state + sum(revenue) & count of users*
#4b. churn by subscription type & sum(revenue) & count of users

In [46]: `df.columns`

Out[46]: Index(['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count'], dtype='str')

In [47]: *#5. ARPU- Average revenue per user*
`arpu=df['monthly_charges'].mean()`
`print(arpu)`

18.847142857142856

In []: `del print`

In []: *#6 Avg customer tenure*
#count of days users has used our service : cancellation date else current date

```
today=pd.Timestamp.today()
df['tenure_days'] = np.where(
    df['cancellation_date'].notna(),

    (df['cancellation_date']-df['subscription_start_date']).dt.days,
    (today- df['subscription_start_date']).dt.days
)
```

```
avg_tenure= df['tenure_days'].mean()
print("Avg Tenure (Days) =", round(avg_tenure),0)
```

```
In [ ]: df.head()
```

```
In [ ]: #7.revenue at Risk- revenue lost from churned uers
revenue_at_risk=df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at risk(Rs k) = ",revenue_at_risk)
```

```
In [ ]: #8. Escalation Rate
escalations_rate=(df['escalations']=='Y').mean()*100
print("escalations rate =", round(escalations_rate,2),"%")
```

```
In [ ]: #9.Avg complaint per user
avg_complaints=df['complaint_count'].sum()/ df['customerid'].nunique()
print("avg_complaints per users =", round(avg_complaints,2))
```

```
In [ ]: df['escalations'].unique()
```

```
In [ ]: df['escalations'].value_counts()
```

```
In [ ]: #10.correalation escalation vs churn

df['escalations'] = np.where(df['escalations'] == 'Y', 1,0) #encoding int to string
corr_df=df[['escalations', 'churn_flag']].dropna()

correlation= corr_df['escalations'].corr(df['churn_flag'])
print("correlation between escalation vs churn is = ", round(correlation,2)*100 )
```

```
In [ ]: #11. create new column- churn risk

conditions=[
    (df['churn_score']<50),
    (df['churn_score']>=50) & (df['churn_score']<70),
    (df['churn_score']>=70)
]

choices=['low', 'med', 'high']
df['churn_risk']=np.select(conditions, choices, default='unknown')
```

```
In [ ]: df[['churn_score','churn_risk' ]].head(10)
```

```
In [ ]:
```

```
In [ ]:
```

```
In [51]: #4. Data visualization
df_visual= df.copy()
```

```
In [52]: df_visual.columns
```

```
Out[52]: Index(['customerid', 'subscription_start_date', 'subscription_type',
               'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
               'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
               'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
               'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
              dtype='str')
```

```
In [53]: df_visual.shape
```

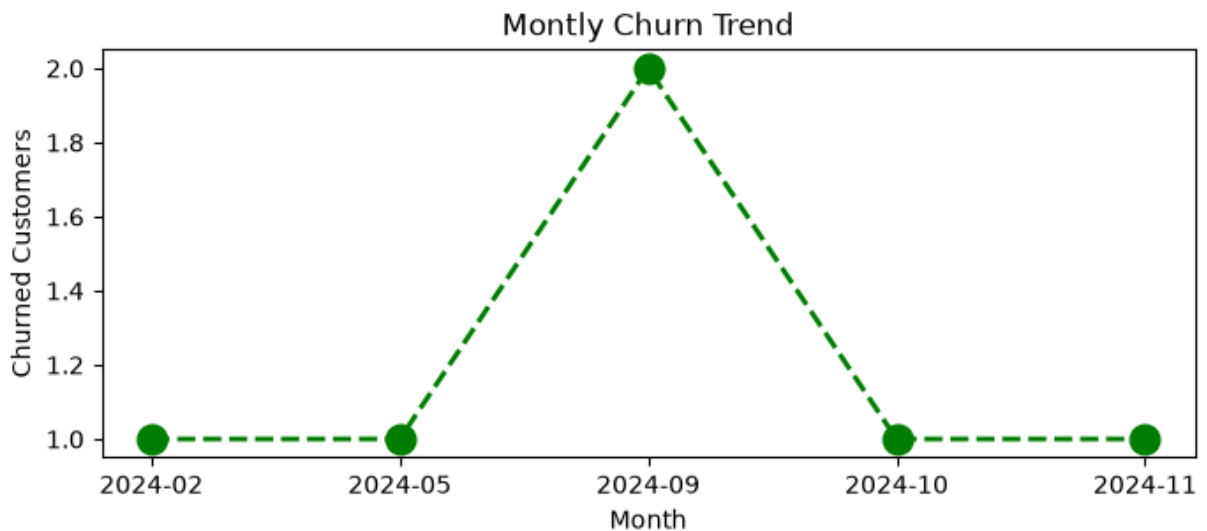
```
Out[53]: (21, 21)
```

```
In [54]: #4.1 Monthly churn trend(Time series KPI)
```

```
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M')
churn_trend=df_visual[df_visual['churn_flag']==1].groupby('cancellation_month').size

plt.figure(figsize= (8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values, color='green', marker=

plt.title('Montly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```

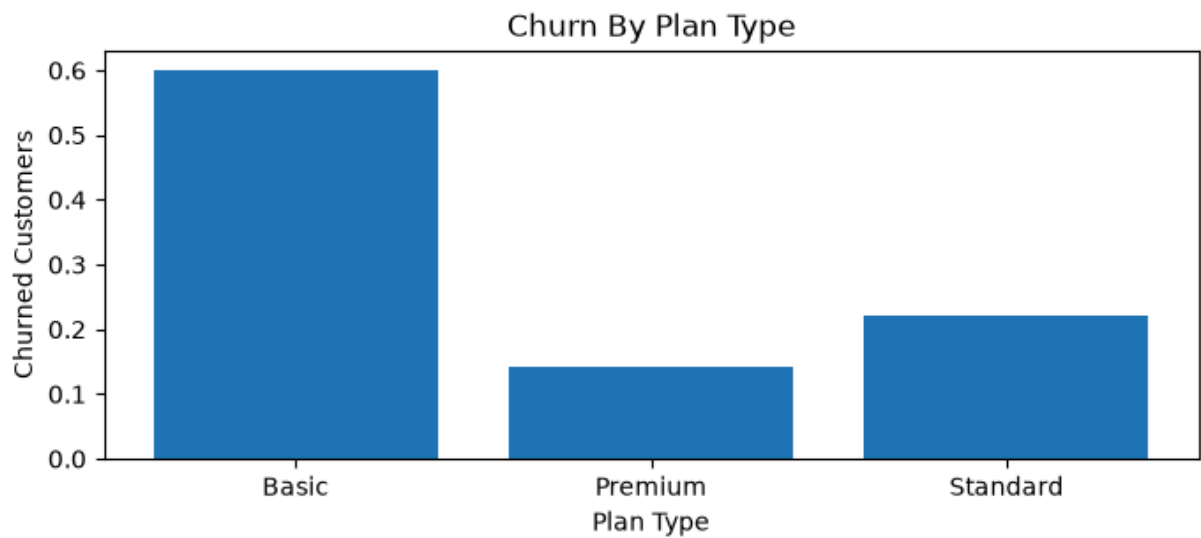


```
In [55]: #4.2 churn by plan type
```

```
churn_flag=df.groupby('plan_type')['churn_flag'].mean()

plt.figure(figsize= (8,3))

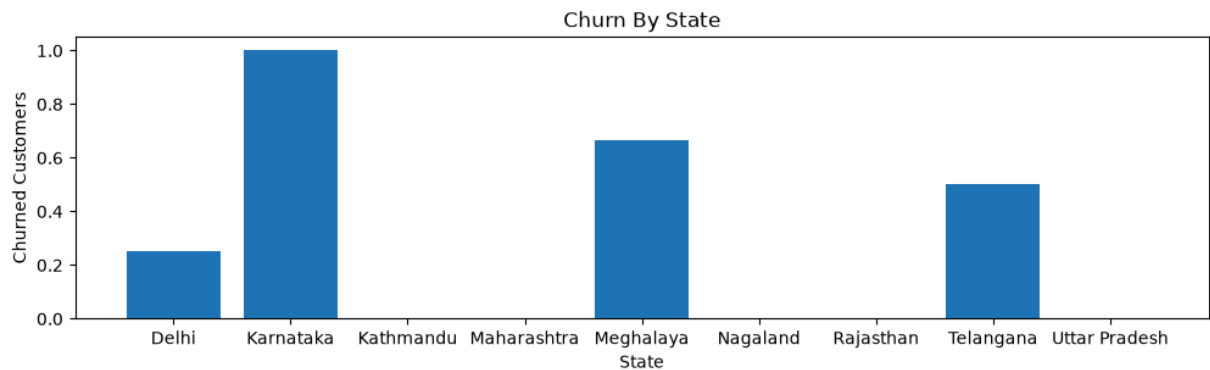
plt.bar(churn_flag.index, churn_flag.values)
plt.title('Churn By Plan Type')
plt.xlabel('Plan Type')
plt.ylabel('Churned Customers')
plt.show()
```



```
In [56]: #4.3 churn by states
churn_state=df.groupby('state')['churn_flag'].mean()

plt.figure(figsize= (12,3))

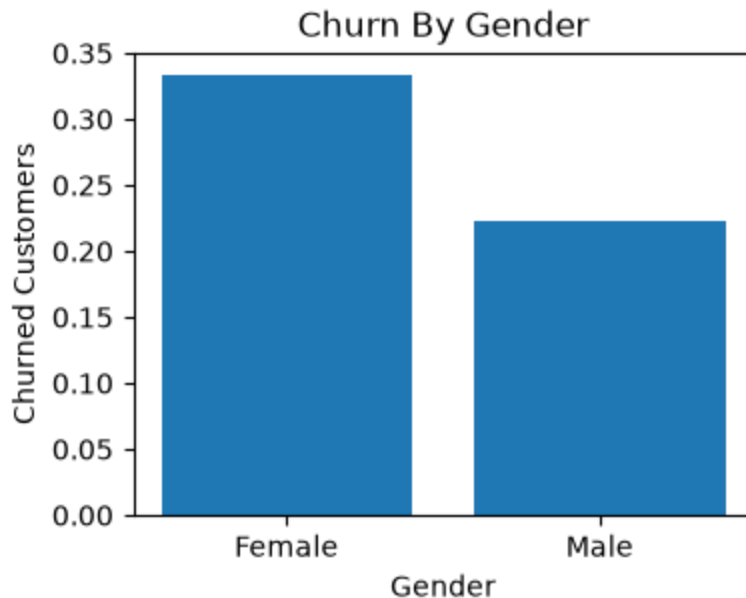
plt.bar(churn_state.index, churn_state.values)
plt.title('Churn By State')
plt.xlabel('State')
plt.ylabel('Churned Customers')
plt.show()
```



```
In [57]: churn_gender=df.groupby('gender')['churn_flag'].mean()

plt.figure(figsize= (4,3))

plt.bar(churn_gender.index, churn_gender.values)
plt.title('Churn By Gender')
plt.xlabel('Gender')
plt.ylabel('Churned Customers')
plt.show()
```



In []:

In []:

In []: *#visualization using seaborn*

In [58]: *#encoding - convert str to numeric so that we can find corr between feature*
`df_visual.columns`

Out[58]: Index(['customerid', 'subscription_start_date', 'subscription_type',
 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
 'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
 'complaint_date', 'escalations', 'csat_score', 'complaint_count',
 'cancellation_month'],
 dtype='str')

In [59]: *#incorrect method of encoding*

```
df_encoded= df[[
    'plan_type', 'contract_type', 'churn_score',
    'churn_flag', 'gender']].head()

categorical_cols= ['plan_type', 'gender', 'contract_type']
for col in categorical_cols:
    df_encoded[col]= df_encoded[col].astype('category').cat.codes
```

In [60]: `df_visual[['plan_type', 'contract_type', 'churn_score',
 'churn_flag', 'gender']].head()`

Out[60]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>
<th>churn_flag</th> <th>gender</th> </thead> <tbody> <th>0</th> <th>1</th>
<th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

Standard	Annual	12	0	Male
----------	--------	----	---	------

Premium	Annual	91	1	Male
---------	--------	----	---	------

Basic	Monthly	34	0	Female
-------	---------	----	---	--------

Premium	Annual	8	0	Male
---------	--------	---	---	------

Standard	Monthly	88	1	Female
----------	---------	----	---	--------

In [61]: df_encoded.head()

Out[61]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>
<th>churn_flag</th> <th>gender</th> </thead> <tbody> <th>0</th> <th>1</th>
<th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

2	0	12	0	1
---	---	----	---	---

1	0	91	1	1
---	---	----	---	---

0	1	34	0	0
---	---	----	---	---

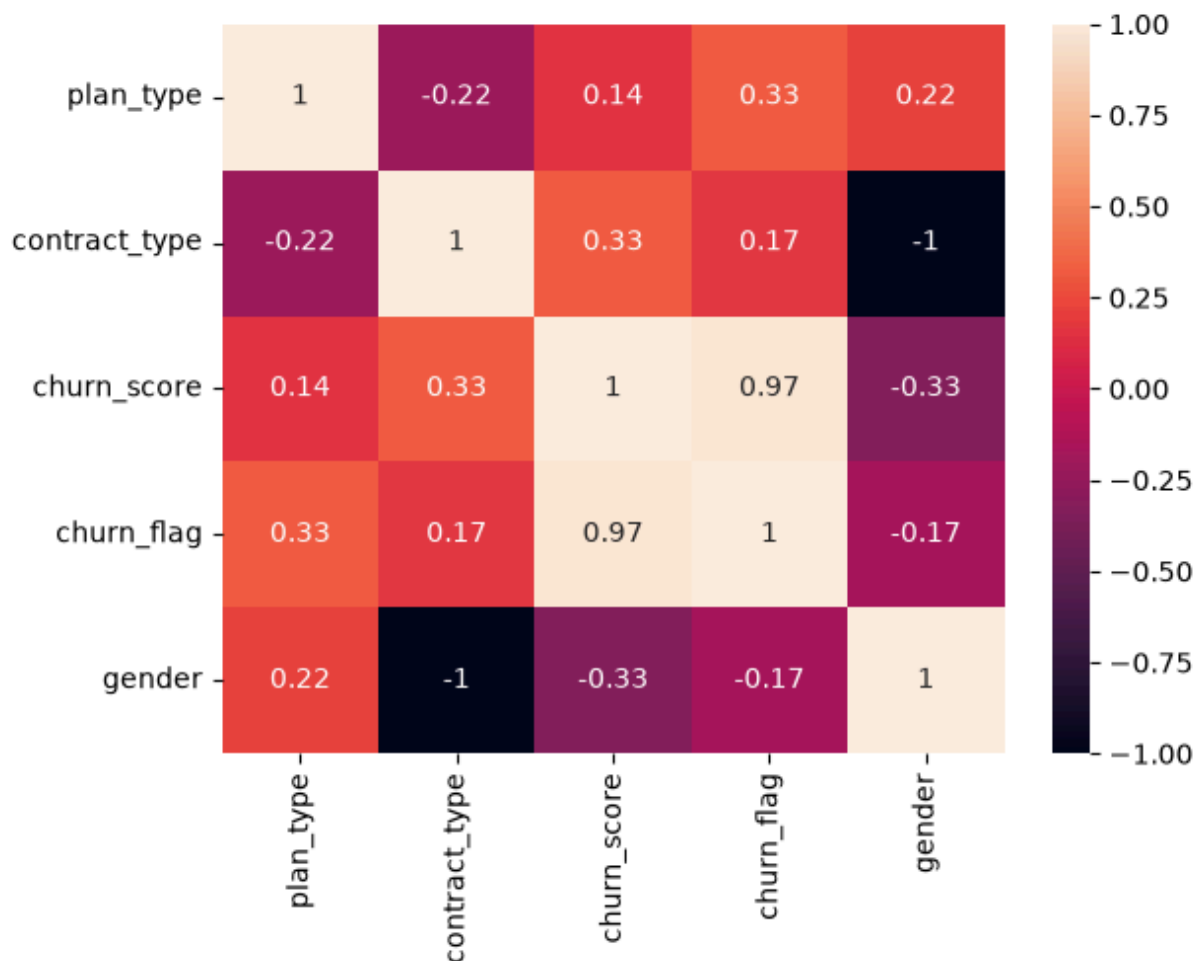
1	0	8	0	1
---	---	---	---	---

2	1	88	1	0
---	---	----	---	---

In [62]: *#heatmap(correlation matrix)*

sns.heatmap(df_encoded.corr(), annot= True)

Out[62]: <Axes: >



```
In [67]: df_visual['gender'].unique()
```

```
Out[67]: <StringArray>
['Male', 'Female']
Length: 2, dtype: str
```

```
In [74]: #correct method of encoding
df_encoded= df[[
    'plan_type', 'contract_type', 'churn_score',
    'churn_flag', 'gender']]

order_mappings= {
    'plan_type' : ['Basic', 'Standard', 'Premium'],
    'contract_type' : ['Monthly', 'Annual'],
    'gender' : ['Female', 'Male']
}

for col, order in order_mappings.items():
    df_encoded[col]= pd.Categorical(df_encoded[col].astype('category'), categories=
```

```
In [76]: df_visual.head()
```

Out[76]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th>
<th>renewal_date</th> <th>plan_type</th> <th>contract_type</th>
<th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th>
<th>cltv</th> <th>...</th> <th>customer_name</th> <th>country</th> <th>state</th>
<th>gender</th> <th>dob</th> <th>complaint_date</th> <th>escalations</th>
<th>csat_score</th> <th>complaint_count</th> <th>cancellation_month</th> </thead>
<tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody>
<tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	...	kesha
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	ragha
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	...	lalit
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	...	moha
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	...	mir

<p>5 rows × 22 columns</p>

In [75]:

df_encoded.head()

Out[75]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th>
<th>churn_flag</th> <th>gender</th> </thead> <tbody> <th>0</th> <th>1</th>
<th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

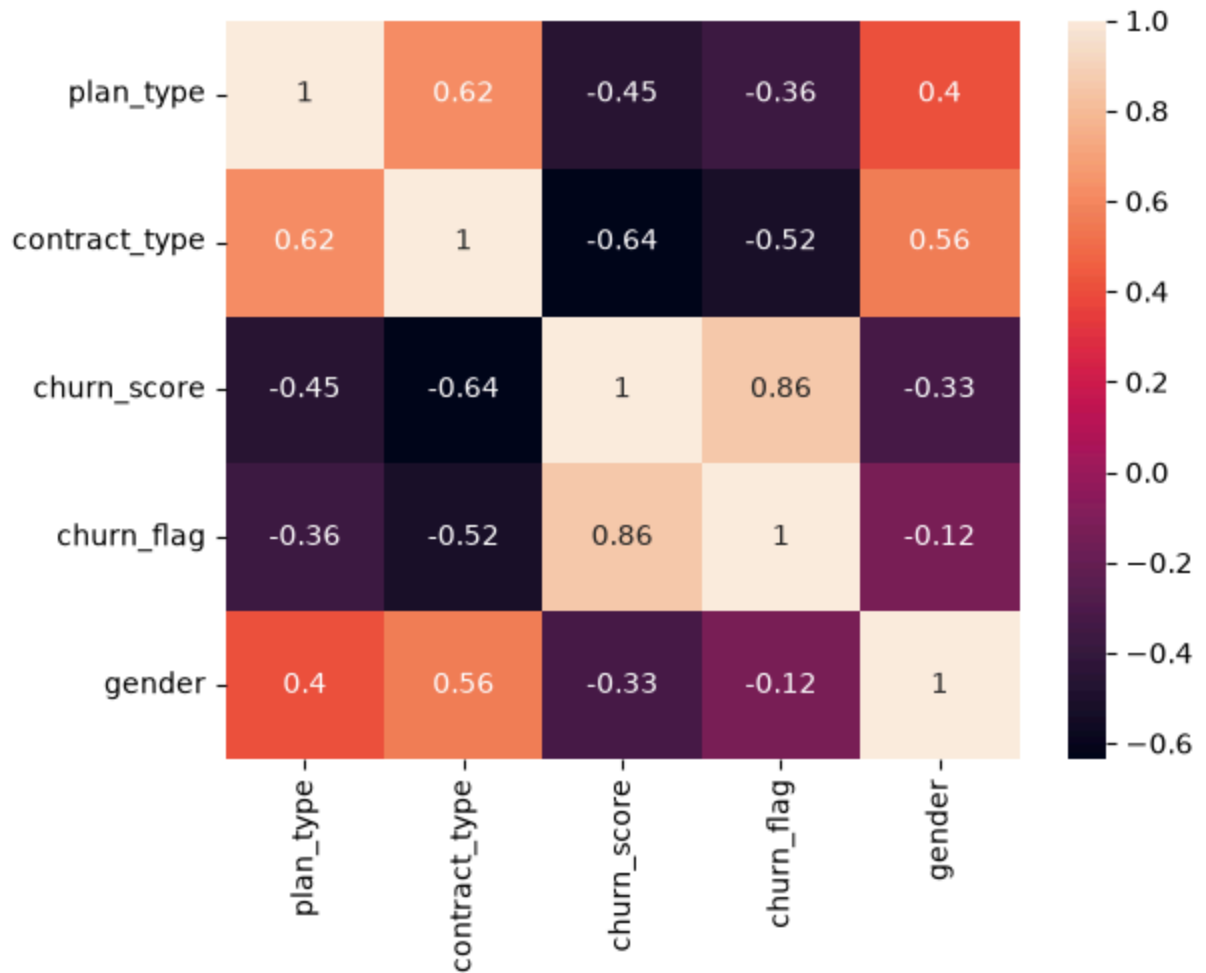
1	1	12	0	1
2	1	91	1	1
0	0	34	0	0
2	1	8	0	1
1	0	88	1	0

In [77]:

sns.heatmap(df_encoded.corr(), annot= True)

Out[77]:

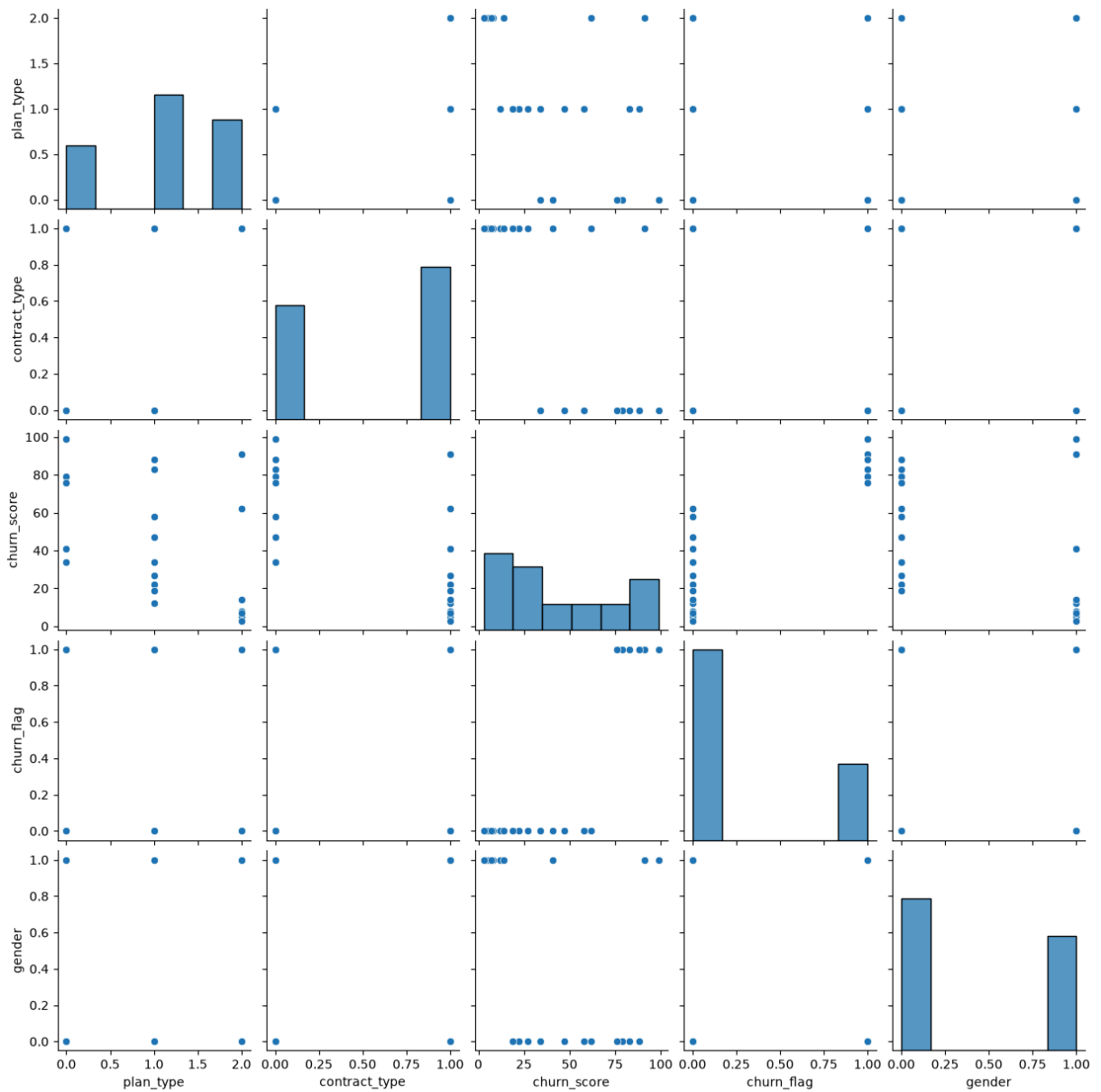
<Axes: >



In [78]: `#pairplot`

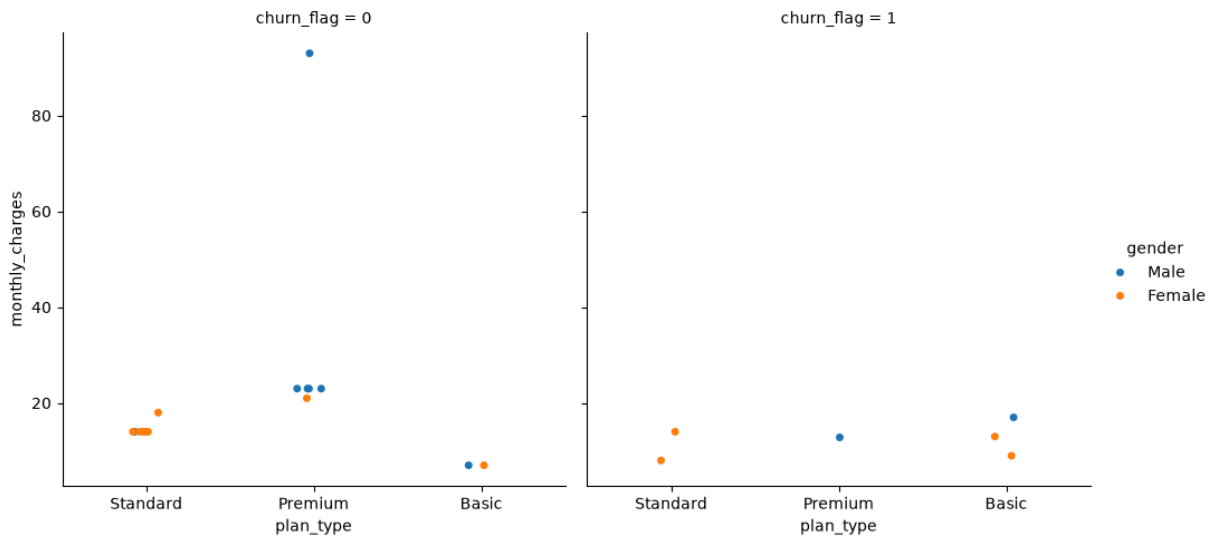
```
sns.pairplot(df_encoded)
```

Out[78]: `<seaborn.axisgrid.PairGrid at 0x2a05a186510>`



```
In [82]: #catplt/Facegrid plot - multi_dim comparison
sns.catplot(data=df_visual,
            x='plan_type',
            y='monthly_charges',
            hue='gender',
            col='churn_flag')
```

```
Out[82]: <seaborn.axisgrid.FacetGrid at 0x2a062e21be0>
```



In [81]: `df_visual.columns`

Out[81]: Index(['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score', 'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob', 'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'cancellation_month'], dtype='str')

In [85]: `pd.pivot_table(df_visual, index='plan_type', values=['monthly_charges', 'customerid', 'churn_flag'], aggfunc={'monthly_charges': 'sum', 'customerid': 'nunique', 'churn_flag': 'mean'})`

Out[85]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>churn_flag</th> <th>customerid</th> <th>monthly_charges</th></thead> <tbody> <tr><th>Basic</th><td>0.600000</td><td>5</td><td>52.95</td></tr> <tr><th>Premium</th><td>0.142857</td><td>7</td><td>218.93</td></tr> <tr><th>Standard</th><td>0.222222</td><td>9</td><td>123.91</td></tr></tbody></table>

In []:

In []:

```
In [ ]: #working with pandas in SQL
conn= sqlite3.connect('test_database.sqlite')

#table details
conn.execute('CREATE TABLE users(first_name TEXT, country TEXT, budget INTEGER)')

#commit and save
conn.commit()
```

```
In [90]: #insert data

cursor = conn.cursor()

#write sql query to insert data in table
cursor.execute(
    """
    INSERT INTO users VALUES
    ('Madhav', 'India', 5000),
    ('Gautam', 'India', 4000),
    ('Fernando', 'France', 10000),
    ('Ajmer', 'Pakistan', 250),
    ('Emma', 'Germany', 15000),
    ('Frank', 'Austria', 8000),
    ('Gautam', 'India', 4000)

    """
)

conn.commit()
```

```
In [93]: #check inserted data
conn= sqlite3.connect('test_database.sqlite')

query= """ Select * from users"""

df_results= pd.read_sql(query,conn)
df_results
```

```
Out[93]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>first_name</th> <th>country</th> <th>budget</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th>
<th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th>
<th>13</th> </tbody> <tbody></tbody>
```

Madhav	India	5000
Gautam	India	4000
Fernando	France	10000
Ajmer	Pakistan	250
Emma	Germany	15000
Frank	Austria	8000
Gautam	India	4000
Madhav	India	5000
Gautam	India	4000
Fernando	France	10000
Ajmer	Pakistan	250
Emma	Germany	15000
Frank	Austria	8000
Gautam	India	4000

```
In [94]: query= '''
SELECT country, SUM(budget) as total_budget
FROM users
GROUP by country'''

df_agg = pd.read_sql(query, conn)
df_agg
```

```
Out[94]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody
tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>
</th> <th>country</th> <th>total_budget</th> </thead> <tbody> <th>0</th>
<th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>
```

Austria	16000
---------	-------

France	20000
--------	-------

Germany	30000
---------	-------

India	26000
-------	-------

Pakistan	500
----------	-----

```
In [95]: conn.close()
```

```
In [ ]:
```